

High Performance Programming [COMTEK3, & ESD3]
Lecture 4: Analytical Modelling of Parallel Systems and Applications

03/2025

Instruction: *The exercise problems are designed to aid your understanding of the concepts covered during the lecture. They are intended to be discussed and solved in a group of 3 – 6 students. The most important part of the exercise is the **learning process**. It is strongly recommended that you avoid the usage of LLMs such as chatGPT and similar AI models to generate solutions **without trying to solve the problems first**.*

Exercise 1: Efficiency of Parallel Sum Algorithm

- (a) Complete the Table 1 for the efficiency of the sum algorithm for adding n numbers on p processing units.
- (b) Make a plot of the speedup and efficiency as a function of the number of processes for the different input sizes.
- (c) Using a balanced binary tree, illustrate the computation performed by the work-efficient parallel prefix sum on $A = [5, 8, -6, 4, 7, 17, -2, 4]$.

n	$p = 1$	$p = 4$	$p = 8$	$p = 16$	$p = 32$
16					
32					
64					
128					
256					
512					

Table 1: Efficiency of sum algorithm

Exercise 2: Speedup using Gustafson's Law

- (a) Using any programming language of your choice, plot the speedup as a function of the number of processors using Gustafson's Law for $\alpha \in \{0, 0.1, 0.2, \dots, 1\}$.
- (b) Comment on your observation.

Exercise 3: Asymptotic Comparison of Algorithms

Consider four algorithms for sorting a given list of n numbers as shown in Table 2.

- (a) Which of these algorithms do you consider 'best' and why?
- (b) Which of these are scalable and which are not?

Exercise 4: Superlinear speedup

Consider the search trees shown in Figure 1, in which the red node represents the solution.

- (a) If a sequential search of the tree is performed using the standard depth-first search (DFS) algorithm, how much time does it take to find the solution if traversing each arc of the tree takes one unit of time?

Algorithm	A_1	A_2	A_3	A_4
p	n^2	$\log_2(n)$	n	$\sqrt{n}$
T_p	1	n	$\sqrt{n}$	$\sqrt{n} \log_2(n)$
S	$n \log_2(n)$	$\log_2(n)$	$\sqrt{n} \log_2(n)$	$\sqrt{n}$
E_p	$\frac{\log_2(n)}{\sqrt{n}}$	1	$\frac{\log_2(n)}{n}$	1
pT_p	$\frac{\sqrt{n}}{n^2}$	$n \log_2(n)$	$n^{1.5}$	$n \log_2(n)$
Ranking				

Table 2: Asymptotic comparison of algorithms

- (b) Assume that the tree is partitioned between two processing elements that are assigned to do the search job, as shown in Figure 1 (right). If both processing elements perform a DFS on their respective halves of the tree, how much time does it take for the solution to be found? What is the speedup? Is there a speedup anomaly? If so, can you explain the anomaly?
- (c) Repeat (a) and (b) for the tree in the Figure 2.

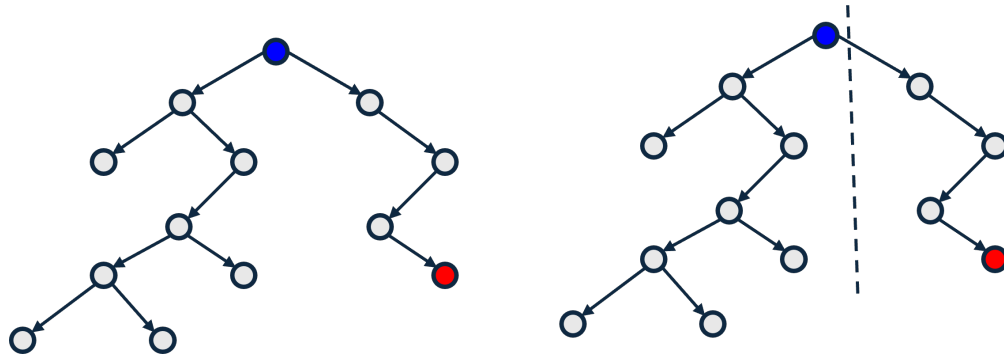


Figure 1: Depth First Search with one processor (left) and 2 processors (right)

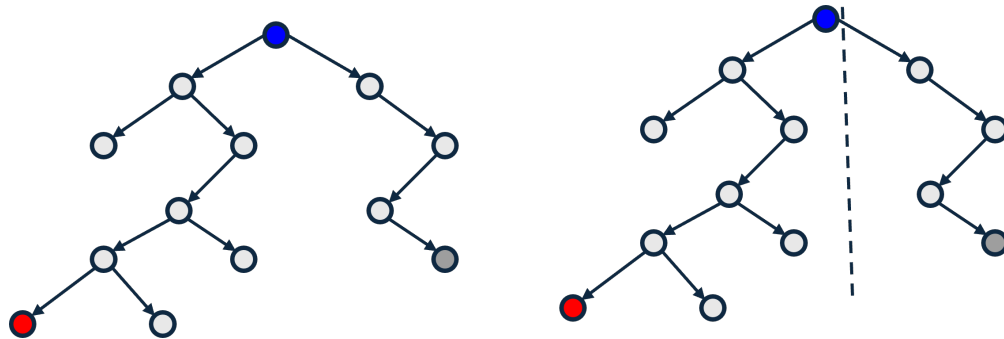


Figure 2: Depth First Search with one processor (left) and 2 processors (right)

Exercise 5: Scaled Speedup

Scaled speedup is defined as the speedup obtained when the problem size is increased linearly

with the number of processing elements. That is, if W is chosen as a base problem size for a single processing element, then:

$$S_{scaled} = \frac{pW}{T_p(pW, p)}$$

- (a) Consider the problem of adding n numbers on p processing elements, and plot the speedup curves, assuming that the base problem for $p = 1$ is that of adding 256 numbers. Use $p = 1, 4, 16, 64, 256$.
- (b) Assume that it takes 10-time units to communicate a number between two processing elements and that it takes one unit of time to add two numbers. Now plot the standard speedup curve for the base problem size and compare it with the scaled speedup curve (*Hint: The parallel runtime $T_p = (n/p - 1) + 11 \log_2(p)$*).

Exercise 6: Application of Parallel Prefix Scan

Parallel prefix scan has many applications in parallel computing.

- (a) Describe how parallel prefix scan can be used in computing the exclusive scan of an array.
- (b) Explain how parallel prefix scan can be applied to solve the problem of parallel radix sorting.
- (c) Discuss how parallel prefix scan can be used to efficiently compute the histogram of a large dataset in parallel.

RAD